

סדנה מתקדמת בתכנות - רשימות מקושרת חד-כיוונית

(1)

בתרגיל זה נשתמש במבנה נתונים של רשימה מקושרת חד-כיוונית. כל איבר (*item*) ברשימה יכיל שדה של מספר שלם (*int*) ושדה מצביע (** struct node*) לאיבר הבא. יש להגדיר את המבנה באופן הבא:

```
typedef struct item {  
    int data;  
    struct node* next;  
} item;
```

1. בניית רשימה ממערך:

כתבו פונקציה המקבלת מערך של מספרים שלמים וגודל המערך, ובונה ממנו רשימה מקושרת אחת בצורה הבאה:

- כל איבר **אי-זוגי** מהמערך יוכנס ל**ראש** הרשימה המקושרת.
- כל איבר **זוגי** מהמערך יוכנס ל**סוף** הרשימה המקושרת.

הפונקציה תעבור על איברי המערך פעם אחת ותחזיר כתובת של ראש הרשימה החדשה.

2. הדפסת רשימה:

כתבו פונקציה כללית להדפסת רשימה מקושרת המציגה את כל ערכי האיברים מהראש ועד הסוף.

3. הוספת איבר לראש הרשימה:

כתבו פונקציה להוספת איבר חדש בראש הרשימה המקושרת. על הפונקציה להציג שתי שיטות לעדכון ראש הרשימה החדש:

- החזרת כתובת הראש החדש באמצעות פקודת `return`.
- עדכון כתובת הראש באמצעות מצביע למצביע המועבר כפרמטר (*by reference*).

4. מחיקת איבר ראשון:

כתבו פונקציה המקבלת את ראש הרשימה, מוחקת את האיבר הראשון (הראש) ומשחררת את הזיכרון שלו. הפונקציה תעדכן את ראש הרשימה

לכתובת האיבר שהיה שני בתור. יש לטפל במקרה קצה שבו **הרשימה ריקה** ואין איברים למחוק.

5. **מחיקת איבר אחרון:**

כתבו פונקציה המקבלת את ראש הרשימה, מוצאת את האיבר האחרון ברשימה, מוחקת ומשחררת את הזיכרון שלו. יש לוודא שהמצביע `next` של האיבר **שלפני האחרון** יעודכן ל-`NULL`. יש לטפל במקרה קצה שבו **הרשימה ריקה** ואין איברים למחוק.

הנחיות נוספות: -לאחר כל עדכון (בנייה, הוספה או מחיקה), יש לקרוא לפונקציית ההדפסה

- בכל השאלות ניתן להניח שיש בזיכרון מספיק מקום להקצאה.
בהצלחה!!